

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

“Информатика”

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

Выполнила:

Студентка гр. 3148 Жук Анастасия



Проверил:

Безруков В. А.

Санкт-Петербург
2023г.

Перевод числа в строку в Assembler с помощью C Код на языке Assembler

C подробными комментариями:

```
.586 ;тип процессора
.MODEL FLAT, STDCALL ;плоская модель памяти
PUBLIC fun1
_DATA SEGMENT
    ;сегмент данных
    strmas db 12 dup(?), 0 ;12 раз повторить по байту и в каждом байте оставить мусор
    dec1 dd 10 ;основание системы счисления
_DATA ENDS
_TEXT SEGMENT
fun1 PROC par1: DWORD ;на вход подается одно число
    ;получить смещение в сегменте данных строки, которой мы выделили память в байтовой
    ;подстроке
    lea EBX, strmas ;указатель на строку (адрес сегмента данных + смещение)
    mov ECX, 12 ;количество повторений цикла
    ;очистка - меняем мусор на пробелы
    metka1:
        mov BYTE PTR [EBX], ' ' ;приведение типа указателя + разыменование
        ;указателя ([ ]) + код пробела
        inc EBX ;адрес увеличивается на 1 байт (переходим к следующему байту)
        Loop metka1 ;уменьшаем значение счетчика
    ;подготовка числа к переводу (переводить в строку можно только положительное
    ;число)
    mov EAX, par1 ;загружаем par1 в регистр-аккумулятор
    push EAX ;кладем в стек копию числа
    OR EAX, EAX ;проверяем на наличие знака (операнд не изменится, а флаг знака
    ;сохранится: 1 - отрицательное, 0 - положительное)
    jns metka2 ;если флаг знака установлен, то делаем следующую команду, иначе
    ;переходим в metka2
    neg EAX ;меняем знак числа, если оно было отрицательное
    ;перевод числа в строку
    metka2:
        XOR EDX, EDX ;обнуляем регистр для точного получения остатка
        div dec1 ;делим на основание системы счисления и получаем остаток
        add DL, '0' ;добавляем код символа "ноль" к остатку и получаем код символа
        ;числа
        dec EBX ;возвращаемся на 1 байт назад
        mov BYTE PTR [EBX], DL ;приведение типа + занесение остатка
        inc ECX ;счетчик внесенных символов (увеличиваем на 1)
        OR EAX, EAX ;проверка на равенство нулю частного
        jnz metka2 ;если результат не 0, то снова делим (начинаем цикл сначала)
    ;установка знака (если нужно)
    pop EAX ;вытаскиваем исходное число из стека и проверяем его знак
    OR EAX, EAX ;проверка на отрицательность
    jns metka3 ;если знак не установлен, то metka3, иначе делаем следующую команду
    dec EBX ;смещение влево
    mov BYTE PTR [EBX], '-' ;записываем код символа "знак"
    inc ECX ;увеличиваем счетчик символов
    ;формирование ответа
    metka3:
        mov EAX, EBX ;возвращаем в регистр-аккумулятор полученную строку
        ret
fun1 ENDP
_TEXT ENDS
END
```

С краткими комментариями (основные этапы):

```
.586 ;тип процессора
.MODEL FLAT, STDCALL ;плоская модель памяти
PUBLIC fun1
_DATA SEGMENT
    ;сегмент данных
    strmas db 12 dup(?), 0
    dec1 dd 10
_DATA ENDS
_TEXT SEGMENT
fun1 PROC par1: DWORD
    ;получить смещение в сегменте данных строки, которой мы выделили память в байтовой
    ;подстроке
    Lea EBX, strmas
    mov ECX, 12
    ;очистка - меняем мусор на пробелы
    metka1:
        mov BYTE PTR [EBX], ' '
        inc EBX
        Loop metka1
    ;подготовка числа к переводу (переводить в строку можно только положительное
    ;число)
    mov EAX, par1
    push EAX
    OR EAX, EAX
    jns metka2
    neg EAX
    ;перевод числа в строку
    metka2:
        XOR EDX, EDX
        div dec1
        add DL, '0'
        dec EBX
        mov BYTE PTR [EBX], DL
        inc ECX
        OR EAX, EAX
        jnz metka2
    ;установка знака (если нужно)
    pop EAX
    OR EAX, EAX
    jns metka3
    dec EBX
    mov BYTE PTR [EBX], '-'
    inc ECX
    ;формирование ответа
    metka3:
        mov EAX, EBX
        ret
fun1 ENDP
_TEXT ENDS
END
```

Программа на C

```
Текст.asm  Lab2Inf_2.cpp  X
Lab2Inf_2  (Глобальная область)

#include <stdio.h>
extern "C" char* _stdcall fun1(int); //на вход функция получает число типа int, а выводит строку типа char

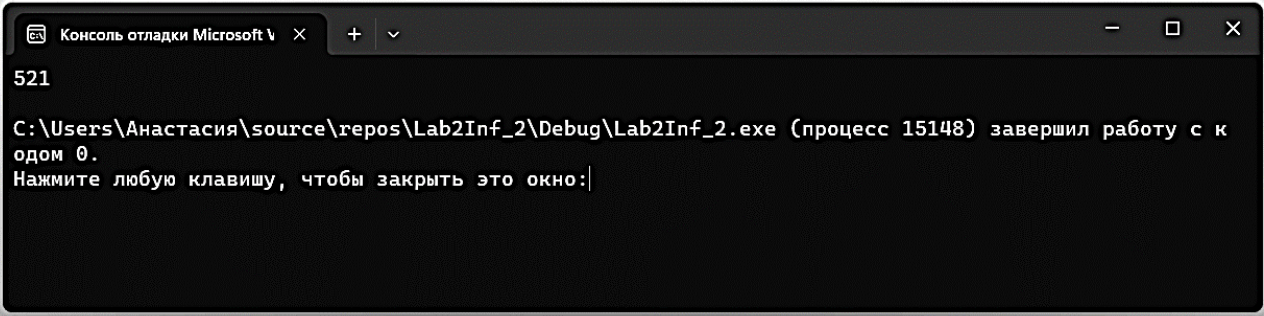
int main()
{
    printf("%s\n", fun1(521)); //выводим результат выполнения функции fun1
    return 0;
}
```

Примеры работы программы

```
Текст.asm  Lab2Inf_2.cpp  X
Lab2Inf_2  (Глобальная область)

#include <stdio.h>
extern "C" char* _stdcall fun1(int); //на вход функция получает число типа int, а выводит строку типа char

int main()
{
    printf("%s\n", fun1(521)); //выводим результат выполнения функции fun1
    return 0;
}
```



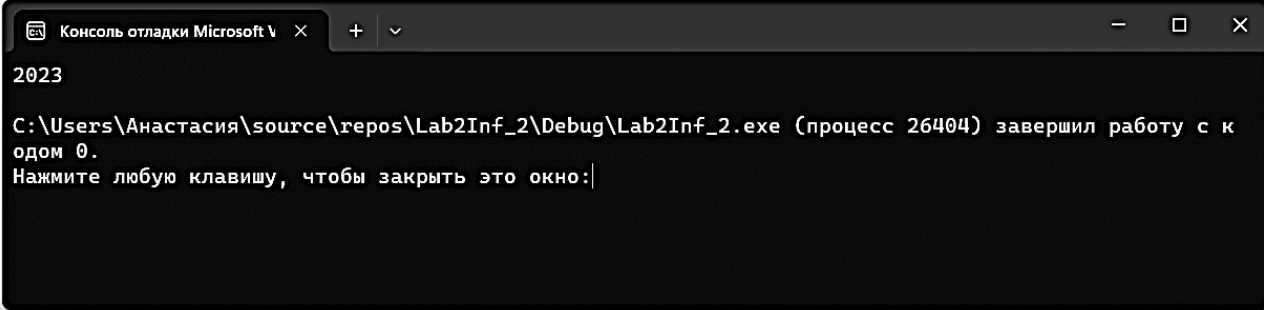
```
Консоль отладки Microsoft V  X  +  v
521

C:\Users\Анастасия\source\repos\Lab2Inf_2\Debug\Lab2Inf_2.exe (процесс 15148) завершил работу с кодом 0.
Нажмите любую клавишу, чтобы закрыть это окно:
```

```
Текст.asm  Lab2Inf_2.cpp  X
Lab2Inf_2  (Глобальная область)  main()

#include <stdio.h>
extern "C" char* _stdcall fun1(int); //на вход функция получает число типа int, а выводит строку типа char

int main()
{
    printf("%s\n", fun1(2023)); //выводим результат выполнения функции fun1
    return 0;
}
```



```
Консоль отладки Microsoft V  X  +  v
2023

C:\Users\Анастасия\source\repos\Lab2Inf_2\Debug\Lab2Inf_2.exe (процесс 26404) завершил работу с кодом 0.
Нажмите любую клавишу, чтобы закрыть это окно:
```